

## HIGH PERFORMANCE COMPUTING

### Assginment-1

**NAME :** Juluri Pavan

**HT No :** 2303A51412

**BATCH :** 21

**Section1:** Section A — SSH Access & Environment Sanity Check (Login Node)

```
%%bash python --  
version
```

Python 3.12.12

```
%%bash which  
python which  
python3
```

```
/usr/local/bin/python  
/usr/bin/python3
```

```
%%bash
```

```
lscpu
```

```
Architecture:                x86_64  
CPU op-mode(s):              32-bit, 64-bit  
Address sizes:               46 bits physical, 48 bits virtual  
Byte Order:                  Little Endian  
CPU(s):                      2  
On-line CPU(s) list:        0,1  
Vendor ID:                   GenuineIntel  
Model name:                  Intel(R) Xeon(R) CPU @ 2.20GHz  
CPU family:                  6  
Model:                       79  
Thread(s) per core:         2  
Core(s) per socket:         1  
Socket(s):                   1  
Stepping:                    0  
BogoMIPS:                    4399.99  
Flags:                       fpu vme de pse tsc msr pae mce cx8  
apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall  
nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid  
tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe  
popcnt aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd  
ibrs ibpb stibp fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm  
rdseed adx smap xsaveopt arat md_clear arch_capabilities  
Hypervisor vendor:           KVM  
Virtualization type:         full
```

|                                          |                                                                                          |
|------------------------------------------|------------------------------------------------------------------------------------------|
| L1d cache:                               | 32 KiB (1 instance)                                                                      |
| L1i cache:                               | 32 KiB (1 instance)                                                                      |
| L2 cache:                                | 256 KiB (1 instance)                                                                     |
| L3 cache:                                | 55 MiB (1 instance)                                                                      |
| NUMA node(s):                            | 1                                                                                        |
| NUMA node0 CPU(s):                       | 0,1                                                                                      |
| Vulnerability Gather data sampling:      | Not affected                                                                             |
| Vulnerability Indirect target selection: | Vulnerable                                                                               |
| Vulnerability Itlb multihit:             | Not affected                                                                             |
| Vulnerability L1tf:                      | Mitigation; PTE Inversion                                                                |
| Vulnerability Mds:                       | Vulnerable; SMT Host state unknown                                                       |
| Vulnerability Meltdown:                  | Vulnerable                                                                               |
| Vulnerability Mmio stale data:           | Vulnerable                                                                               |
| Vulnerability Reg file data sampling:    | Not affected                                                                             |
| Vulnerability Retbleed:                  | Vulnerable                                                                               |
| Vulnerability Spec rstack overflow:      | Not affected                                                                             |
| Vulnerability Spec store bypass:         | Vulnerable                                                                               |
| Vulnerability Spectre v1:                | Vulnerable: __user pointer sanitization and usercopy barriers only; no swaps barriers    |
| Vulnerability Spectre v2:                | Vulnerable; IBPB: disabled; STIBP: disabled; PBR SB-eIBRS: Not affected; BHI: Vulnerable |
| Vulnerability Srbds:                     | Not affected                                                                             |
| Vulnerability Tsa:                       | Not affected                                                                             |
| Vulnerability Tsx async abort:           | Vulnerable                                                                               |

```
%%bash cat
```

```
/proc/cpuinfo
```

```
processor : 0
vendor_id : GenuineIntel
cpu family : 6
model      : 79
model name : Intel(R) Xeon(R) CPU @ 2.20GHz
stepping   : 0
microcode  : 0xffffffff
cpu MHz    : 2199.998 cache size
           : 56320 KB
physical id : 0
siblings   : 2
core id    : 0
cpu cores  : 1
apicid     : 0
initial apicid : 0
fpu        : yes
fpu_exception : yes
cpuid level : 13
wp         : yes
```

flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat  
pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm  
constant\_tsc rep\_good nopl xtopology nonstop\_tsc cpuid tsc\_known\_freq pni  
pclmulqdq ssse3 fma cx16 pcid sse4\_1 sse4\_2 x2apic movbe popcnt aes xsave avx  
f16c rdrand hypervisor lahf\_lm abm 3dnowprefetch ssbd ibrs ibpb stibp  
fsgsbase tsc\_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed adx smap  
xsaveopt arat md\_clear arch\_capabilities  
bugs : cpu\_meltdown spectre\_v1 spectre\_v2 spec\_store\_bypass l1tf mds  
swapgs taa mmio\_stale\_data retbleed bhi its  
bogomips : 4399.99 clflush size : 64  
cache\_alignment : 64 address sizes : 46 bits  
physical, 48 bits virtual power management:

processor : 1 vendor\_id :

GenuineIntel

cpu family : 6

model : 79

model name : Intel(R) Xeon(R) CPU @ 2.20GHz

stepping : 0

microcode : 0xffffffff

cpu MHz : 2199.998

cache size : 56320 KB

physical id : 0

siblings : 2

core id : 0

cpu cores : 1

apicid : 1 initial

apicid : 1

fpu : yes

fpu\_exception : yes

cpuid level : 13

wp : yes

flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat

pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm

constant\_tsc rep\_good nopl xtopology nonstop\_tsc cpuid tsc\_known\_freq pni

pclmulqdq ssse3 fma cx16 pcid sse4\_1 sse4\_2 x2apic movbe popcnt aes xsave avx

f16c rdrand hypervisor lahf\_lm abm 3dnowprefetch ssbd ibrs ibpb stibp

fsgsbase tsc\_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed adx smap

xsaveopt arat md\_clear arch\_capabilities

bugs : cpu\_meltdown spectre\_v1 spectre\_v2 spec\_store\_bypass l1tf mds

swapgs taa mmio\_stale\_data retbleed bhi its

bogomips : 4399.99

clflush size : 64 cache\_alignment : 64 address

sizes : 46 bits physical, 48 bits virtual power

management:

```
%%bash
```

```
nproc
```

```
2
```

```
%%bash
```

```
echo 'print("Hello from the HPC cluster!")' > sanity_check.py
```

```
cat sanity_check.py print("Hello from the HPC cluster!")
```

```
%%bash python
```

```
sanity_check.py
```

```
Hello from the HPC cluster!
```

## Section 2: Section B — First Batch Job: Serial Python Script via Slurm/PBS

```
%%bash
```

```
cat <<EOF > submit_job.sh
```

```
#!/bin/bash
```

```
#SBATCH --job-name=MyPythonJob
```

```
#SBATCH --output=slurm-%j.out
```

```
#SBATCH --error=slurm-%j.err
```

```
#SBATCH --ntasks=1
```

```
#SBATCH --time=00:01:00
```

```
echo "Starting Python script..."
```

```
python sanity_check.py echo
```

```
"Python script finished." EOF
```

```
%%bash cat
```

```
submit_job.sh
```

```
#!/bin/bash
```

```
#SBATCH --job-name=MyPythonJob
```

```
#SBATCH --output=slurm-%j.out
```

```
#SBATCH --error=slurm-%j.err
```

```
#SBATCH --ntasks=1
```

```
#SBATCH --time=00:01:00
```

```
echo "Starting Python script..."
```

```
python sanity_check.py echo
```

```
"Python script finished."
```

```
%%bash bash submit_job.sh > slurm-12345.out 2> slurm-  
12345.err
```

```
%%bash ls -l slurm-12345.out slurm-  
12345.err
```

```
-rw-r--r-- 1 root root 0 Jan 6 04:49 slurm-12345.err
```

```
-rw-r--r-- 1 root root 78 Jan 6 04:49 slurm-12345.out
```

```
%%bash cat slurm-  
12345.out
```

```
Starting Python script...  
Hello from the HPC cluster!  
Python script finished.
```

```
%%bash cat slurm-  
12345.err
```

### Section 3: Section C — Interactive Compute Session (salloc/srun or qsub -l)

```
%%bash  
cat <<EOF > run_interactive_python.sh  
#!/bin/bash  
#SBATCH --job-name=InteractivePython  
#SBATCH --output=interactive-%j.out  
#SBATCH --error=interactive-%j.err  
#SBATCH --ntasks=1  
#SBATCH --time=00:01:00
```

```
echo "Launching simulated interactive session..."  
python sanity_check.py  
echo "Simulated interactive session finished."  
EOF
```

```
# Run the script and redirect output to files  
bash run_interactive_python.sh > interactive-12346.out 2> interactive-  
12346.err echo "Script run_interactive_python.sh created and  
  
executed."
```

Script run\_interactive\_python.sh created and executed.

```
%%bash  
ls -l interactive-12346.out interactive-12346.err  
-rw-r--r-- 1 root root 0 Jan  6 05:03 interactive-12346.err  
-rw-r--r-- 1 root root 111 Jan  6 05:03 interactive-12346.out  
%%bash cat interactive-  
12346.out
```

```
Launching simulated interactive session...  
Hello from the HPC cluster!  
Simulated interactive session finished.
```

```
%%bash  
cat interactive-12346.err
```

### Section 4: Section D — Job Arrays for Parameter Sweep (Multiple Serial Runs)

```

%%bash
cat <<EOF > run_monte_carlo.py
import sys import random

def estimate_pi(sample_size):
    inside_circle = 0 for _ in
    range(sample_size): x =
    random.uniform(0, 1) y =
    random.uniform(0, 1)
    distance = x**2 + y**2 if
    distance <= 1:
        inside_circle += 1

    pi_estimate = 4 * inside_circle / sample_size
    return pi_estimate

if __name__ == '__main__':
    if len(sys.argv) != 2:
        print("Usage: python run_monte_carlo.py <sample_size>")
        sys.exit(1)

    sample_size = int(sys.argv[1]) pi_est =
estimate_pi(sample_size) print(f"Sample Size: {sample_size}, Pi
Estimate: {pi_est}") EOF cat run_monte_carlo.py

import sys
import random

def estimate_pi(sample_size):
    inside_circle = 0 for _ in
    range(sample_size): x =
    random.uniform(0, 1) y =
    random.uniform(0, 1)
    distance = x**2 + y**2 if
    distance <= 1:
        inside_circle += 1

    pi_estimate = 4 * inside_circle / sample_size
    return pi_estimate

if __name__ == '__main__':
    if len(sys.argv) != 2:
        print("Usage: python run_monte_carlo.py <sample_size>")
        sys.exit(1)
    sample_size = int(sys.argv[1]) pi_est =
    estimate_pi(sample_size) print(f"Sample Size:
    {sample_size}, Pi Estimate: {pi_est}")

```

Now, let's create the Slurm batch script `submit_array_job.sh`. This script will define an array job that runs `run_monte_carlo.py` with different sample sizes based on the `SLURM_ARRAY_TASK_ID` environment variable.

```
%%bash
cat <<EOF > submit_array_job.sh
#!/bin/bash
#SBATCH --job-name=MonteCarloArray
#SBATCH --output=array-job-%A_%a.out
#SBATCH --error=array-job-%A_%a.err
#SBATCH --ntasks=1
#SBATCH --time=00:02:00
#SBATCH --array=0-2 # This creates 3 tasks: 0, 1, 2

# Calculate sample size based on array task ID
# Task 0 -> 1000, Task 1 -> 2000, Task 2 -> 3000
SAMPLE_SIZE=$((1000 * (SLURM_ARRAY_TASK_ID + 1)))

echo "Starting Monte Carlo simulation for task $SLURM_ARRAY_TASK_ID with
sample size $SAMPLE_SIZE" python run_monte_carlo.py $SAMPLE_SIZE
echo "Finished Monte Carlo simulation for task $SLURM_ARRAY_TASK_ID"

EOF cat submit_array_job.sh

#!/bin/bash
#SBATCH --job-name=MonteCarloArray
#SBATCH --output=array-job-%A_%a.out
#SBATCH --error=array-job-%A_%a.err
#SBATCH --ntasks=1
#SBATCH --time=00:02:00
#SBATCH --array=0-2 # This creates 3 tasks: 0, 1, 2

# Calculate sample size based on array task ID
# Task 0 -> 1000, Task 1 -> 2000, Task 2 -> 3000
SAMPLE_SIZE=1000

echo "Starting Monte Carlo simulation for task with sample size " python
run_monte_carlo.py
echo "Finished Monte Carlo simulation for task "
Since we are in a simulated environment, we will manually simulate the execution of the
array job by looping through the array task IDs and running the script for each. We'll
capture the output and error for each simulated task.

%%bash

JOB_ID=98765 # Simulate a job ID

for SLURM_ARRAY_TASK_ID in 0 1 2;
```

```
do
    SAMPLE_SIZE=$((1000 * (SLURM_ARRAY_TASK_ID + 1)))
    OUTPUT_FILE="array-job- $\{JOB\_ID\}$ - $\{SLURM\_ARRAY\_TASK\_ID\}$ .out"
    ERROR_FILE="array-job- $\{JOB\_ID\}$ - $\{SLURM\_ARRAY\_TASK\_ID\}$ .err"

    echo "Starting Monte Carlo simulation for task  $\{SLURM\_ARRAY\_TASK\_ID\}$  with
sample size  $\{SAMPLE\_SIZE\}$ " > " $\{OUTPUT\_FILE\}$ "
    python run_monte_carlo.py  $\{SAMPLE\_SIZE\}$  >> " $\{OUTPUT\_FILE\}$ " 2>>
 $\{ERROR\_FILE\}$  echo "Finished Monte Carlo simulation for task
 $\{SLURM\_ARRAY\_TASK\_ID\}$ " >>
 $\{OUTPUT\_FILE\}$ 

    echo "Simulated execution for task  $\{SLURM\_ARRAY\_TASK\_ID\}$ . Output in
 $\{OUTPUT\_FILE\}$ , Error in  $\{ERROR\_FILE\}$ "
done
```

```
Simulated execution for task 0. Output in array-job-98765_0.out, Error in
array-job-98765_0.err
Simulated execution for task 1. Output in array-job-98765_1.out, Error in
array-job-98765_1.err
Simulated execution for task 2. Output in array-job-98765_2.out, Error in
array-job-98765_2.err
```

Let's list the generated output and error files and then examine their contents.

```
%%bash
ls -l array-job-*.out array-job-*.err
-rw-r--r-- 1 root root 0 Jan 6 05:06 array-job-98765_0.err
-rw-r--r-- 1 root root 146 Jan 6 05:06 array-job-98765_0.out
-rw-r--r-- 1 root root 0 Jan 6 05:06 array-job-98765_1.err
-rw-r--r-- 1 root root 146 Jan 6 05:06 array-job-98765_1.out
-rw-r--r-- 1 root root 0 Jan 6 05:06 array-job-98765_2.err
-rw-r--r-- 1 root root 159 Jan 6 05:06 array-job-98765_2.out
%%bash
cat array-job-98765_0.out
cat array-job-98765_1.out
cat array-job-98765_2.out
```

```
Starting Monte Carlo simulation for task 0 with sample size 1000
Sample Size: 1000, Pi Estimate: 3.108
Finished Monte Carlo simulation for task 0
Starting Monte Carlo simulation for task 1 with sample size 2000
Sample Size: 2000, Pi Estimate: 3.086
Finished Monte Carlo simulation for task 1
Starting Monte Carlo simulation for task 2 with sample size 3000
Sample Size: 3000, Pi Estimate: 3.1226666666666665
Finished Monte Carlo simulation for task 2
```



```
%%bash cat array-job-  
98765_0.err cat array-  
job-98765_1.err cat  
array-job-98765_2.err
```

Section 5: Section E — Modules/Venv, Scratch I/O, and Resource Flags

```
%%bash  
cat <<EOF > scratch_io_script.py  
import sys import os  
  
def main():  
    if len(sys.argv) != 2: print("Usage: python scratch_io_script.py  
    <scratch_directory_path>") sys.exit(1) scratch_dir = sys.argv[1]  
  
    # Create the scratch directory if it doesn't exist  
    os.makedirs(scratch_dir, exist_ok=True) output_file_path =  
    os.path.join(scratch_dir, "output.txt")  
  
    # Write Python version and a message to the file  
    with open(output_file_path, "w") as f:  
        f.write(f"Hello from scratch I/O!\n")  
        f.write(f"Python Version: {sys.version}\n")  
  
    print(f"Output written to {output_file_path}")  
  
if __name__ == '__main__':  
    main() EOF cat  
scratch_io_script.py  
  
import sys  
import os  
  
def main():  
    if len(sys.argv) != 2:  
        print("Usage: python scratch_io_script.py  
        <scratch_directory_path>") sys.exit(1) scratch_dir = sys.argv[1]  
  
    # Create the scratch directory if it doesn't exist  
    os.makedirs(scratch_dir, exist_ok=True) output_file_path  
    = os.path.join(scratch_dir, "output.txt")  
  
    # Write Python version and a message to the file  
    with open(output_file_path, "w") as f:  
        f.write(f"Hello from scratch I/O!\n")
```

```

        f.write(f"Python Version: {sys.version}\n")

    print(f"Output written to {output_file_path}")

if __name__ == '__main__':
    main()

%%bash

# 1. Create a Python virtual environment named 'my_venv'
echo "Creating virtual environment 'my_venv'..." python3
-m venv my_venv

# 2. Activate the 'my_venv' virtual environment
# Note: Activation typically modifies the shell's PATH and prompts. # In a
single bash cell, we can source it and then run commands within the same
shell context. echo "Activating virtual environment 'my_venv'..." source
my_venv/bin/activate

# 3. Install a dummy package (e.g., requests) into the activated virtual
environment
echo "Installing 'requests' package..."
pip install requests

# 4. Verify that the virtual environment is active and the package is
available
echo "Verifying virtual environment and package installation..." python -c
"import sys; print(f'Python executable: {sys.executable}'); import requests;
print('requests package successfully imported.')"

# Deactivate the virtual environment to clean up for subsequent steps if
needed deactivate
echo "Virtual environment deactivated."

Creating virtual environment 'my_venv'...
Activating virtual environment 'my_venv'...
Installing 'requests' package...
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in

```

```
/usr/local/lib/python3.12/dist-packages (from requests) (2025.11.12)
Verifying virtual environment and package installation... Python
executable: /usr/bin/python3 requests package successfully imported.
Virtual environment deactivated.
```

```
Error: Command '['/content/my_venv/bin/python3', '-m', 'ensurepip', '-
upgrade', '--default-pip']' returned non-zero exit status 1. bash: line
10: my_venv/bin/activate: No such file or directory bash: line 21:
deactivate: command not found
```

```
%%bash set -e # Exit immediately if a command exits with a non-zero
status.
```

```
# Clean up any previous attempts
```

```
echo "Cleaning up 'my_venv' directory if it exists..."
rm -rf my_venv
```

```
# 1. Create a Python virtual environment named 'my_venv' without installing
pip initially
```

```
echo "Creating virtual environment 'my_venv' without default pip..."
python3 -m venv --without-pip my_venv
```

```
# 2. Activate the 'my_venv' virtual environment
```

```
echo "Activating virtual environment 'my_venv'..."
. my_venv/bin/activate # Use '.' for sourcing, which is POSIX compliant
```

```
# 3. Manually install pip into the virtual environment
```

```
echo "Installing pip into 'my_venv' using get-pip.py..."
curl -sS https://bootstrap.pypa.io/get-pip.py | python
```

```
# 4. Install a dummy package (e.g., requests) into the activated virtual
environment
```

```
echo "Installing 'requests' package..."
pip install requests
```

```
# 5. Verify that the virtual environment is active and the package is
available
```

```
echo "Verifying virtual environment and package installation..." python -c
"import sys; print(f'Python executable: {sys.executable}'); import requests;
print('requests package successfully imported.')"

```

```
# Deactivate the virtual environment to clean up for subsequent steps if
needed deactivate echo "Virtual environment deactivated."
```

```
Cleaning up 'my_venv' directory if it exists...
```

```
Creating virtual environment 'my_venv' without default pip...
```

```
Activating virtual environment 'my_venv'...
```

```
Installing pip into 'my_venv' using get-pip.py...
```

```

Collecting pip
  Downloading pip-25.3-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-25.3-py3-none-any.whl (1.8 MB)
----- 1.8/1.8 MB 16.8 MB/s 0:00:00
Installing collected packages: pip
Successfully installed pip-25.3
Installing 'requests' package...
Collecting requests
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.4-cp312-
cp312manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.met
adata
(37 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Downloading urllib3-2.6.2-py3-none-any.whl.metadata (6.6 kB)
Collecting certifi>=2017.4.17 (from requests)
  Downloading certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.32.5-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.4-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (153 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading urllib3-2.6.2-py3-none-any.whl (131 kB)
Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi,
requests

Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idna-3.11
requests-2.32.5 urllib3-2.6.2
Verifying virtual environment and package installation...
Python executable: /content/my_venv/bin/python requests
package successfully imported. Virtual environment
deactivated.

```

```

%%bash
cat <<EOF > submit_venv_job.sh
#!/bin/bash
#SBATCH --job-name=VenvTest
#SBATCH --output=slurm-venv-%j.out
#SBATCH --error=slurm-venv-%j.err
#SBATCH --ntasks=1 #SBATCH --time=00:05:00 echo

"Starting Slurm job with virtual environment..."

# Activate the virtual environment
source my_venv/bin/activate

```

```

# Verify Python executable and version within the activated venv
python -c "import sys; print(f'Python executable: {sys.executable}');
print(f'Python version: {sys.version}')"

# Execute the Python script with a simulated scratch directory
python scratch_io_script.py scratch_data

# Deactivate the virtual environment (optional, but good practice)
deactivate

echo "Slurm job with virtual environment finished."

EOF cat submit_venv_job.sh

#!/bin/bash
#SBATCH --job-name=VenvTest
#SBATCH --output=slurm-venv-%j.out
#SBATCH --error=slurm-venv-%j.err
#SBATCH --ntasks=1 #SBATCH --time=00:05:00 echo
"Starting Slurm job with virtual environment..."

# Activate the virtual environment
source my_venv/bin/activate

# Verify Python executable and version within the activated venv
python -c "import sys; print(f'Python executable: {sys.executable}');
print(f'Python version: {sys.version}')"

# Execute the Python script with a simulated scratch directory
python scratch_io_script.py scratch_data

# Deactivate the virtual environment (optional, but good practice)
deactivate
echo "Slurm job with virtual environment finished."

%%bash

JOB_ID=$(date +%s) # Simulate a unique job ID

# Simulate Slurm execution by running the script and redirecting output
bash submit_venv_job.sh > slurm-venv-${JOB_ID}.out 2> slurm-venv-
${JOB_ID}.err

echo "Simulated execution of submit_venv_job.sh. Output in slurm-
venv${JOB_ID}.out, Error in slurm-venv-${JOB_ID}.err"

Simulated execution of submit_venv_job.sh. Output in slurm-venv-
1767676308.out, Error in slurm-venv-1767676308.err

```

```
%%bash
```

```
ls -l slurm-venv-*.out slurm-venv-*.err
```

```
-rw-r--r-- 1 root root 0 Jan 6 05:11 slurm-venv-1767676308.err
```

```
-rw-r--r-- 1 root root 248 Jan 6 05:11 slurm-venv-1767676308.out
```

```
%%bash
```

```
JOB_ID=$(ls slurm-venv-*.out | head -n 1 | sed 's/slurm-venv-//;s/\.out//')
```

```
cat slurm-venv-${JOB_ID}.out cat scratch_data/output.txt
```

```
Starting Slurm job with virtual environment...
```

```
Python executable: /content/my_venv/bin/python
```

```
Python version: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
```

```
Output written to scratch_data/output.txt Slurm job with virtual  
environment finished.
```

```
Hello from scratch I/O!
```

```
Python Version: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
```